

Bind Mount Demonstration in Linux

`mount --bind` using WSL2

Prashant Jadav

Fundamentals of Operating System Design (FOSD)

Learning Objectives

- Understand the concept of a bind mount.
- Understand the difference between a bind mount and copying files.
- Create a bind mount using `mount --bind`.
- Verify a bind mount using `findmnt`.
- Demonstrate that both paths access the same underlying files.
- Remove the bind mount using `umount`.

What is a Bind Mount?

Definition

A bind mount makes an existing directory or file accessible at another location in the Linux directory hierarchy.

- No files are copied.
- No new filesystem is created.
- Both paths refer to the same underlying data.
- The destination directory becomes another access path to the source.

Example

`~/source` can also be accessed through `~/mountpoint`.

Before the Bind Mount

<code>~/source</code>	<code>~/mountpoint</code>
<code>test.txt</code>	Empty directory

Initial State

`~/source` contains the file, while `~/mountpoint` is simply another empty directory.

Step 1: Create Source and Mount Point

```
mkdir ~/source ~/mountpoint
```

- ~/source: directory containing the original files.
- ~/mountpoint: directory where the source will be exposed.

Check the directories:

```
ls -ld ~/source ~/mountpoint
```

Step 2: Create a File in the Source

```
echo "Hello File System" > ~/source/test.txt
```

Check the file:

```
ls -l ~/source  
cat ~/source/test.txt
```

Expected content:

```
Hello File System
```

Observation

The file currently exists only through the ~/source path.

Step 3: Create the Bind Mount

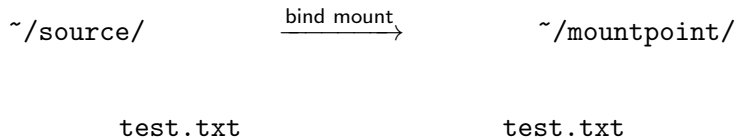
```
sudo mount --bind ~/source ~/mountpoint
```

- `sudo`: execute with root privileges.
- `mount`: attach something to the directory hierarchy.
- `--bind`: use an existing directory as the source.
- `~/source`: source directory.
- `~/mountpoint`: destination mount point.

Key Idea

The contents of `~/source` are now accessible through `~/mountpoint`.

Conceptual View



Important

This is not a copy operation. Both paths provide access to the same underlying files.

Step 4: Verify the Bind Mount

Use:

```
findmnt ~/mountpoint
```

You can also inspect all mounts:

```
findmnt
```

What to observe

The output shows `~/mountpoint` as a mounted target associated with the source directory.

Step 5: Access the File Through the Mount Point

```
ls ~/mountpoint
```

Expected output:

```
test.txt
```

Now read the file:

```
cat ~/mountpoint/test.txt
```

Output:

```
Hello File System
```

Observation

The file created under ~/source is accessible through ~/mountpoint.

Step 6: Demonstrate Shared Data

Create another file through the mount point:

```
echo "Created through mountpoint" > ~/mountpoint/file2.txt
```

Now check the source directory:

```
ls ~/source
```

Expected:

```
file2.txt  
test.txt
```

Conclusion

A file created through one path is immediately visible through the other path because both paths refer to the same underlying data.

Step 7: Remove the Bind Mount

First move outside the mount point if necessary:

```
cd ~
```

Then unmount:

```
sudo umount ~/mountpoint
```

- The bind relationship is removed.
- The source directory is not deleted.
- The files remain in ~/source.

Verify After Unmounting

Check the source:

```
ls ~/source
```

The files should still be present.

Now check the mount point:

```
ls ~/mountpoint
```

The directory is empty again.

Important

`umount` removes the mount relationship; it does not delete the source data.

Complete Demonstration

```
mkdir ~/source ~/mountpoint

echo "Hello File System" > ~/source/test.txt

sudo mount --bind ~/source ~/mountpoint

findmnt ~/mountpoint

ls ~/mountpoint

cat ~/mountpoint/test.txt

echo "Created through mountpoint" > ~/mountpoint/file2.txt

ls ~/source

cd ~
```

Bind Mount vs File Copy

	Bind Mount	Copy
Creates duplicate files	No	Yes
Uses same underlying data	Yes	No
Changes visible through both paths	Yes	No
Requires mount operation	Yes	No
Creates new filesystem	No	No

Bind Mount vs Loop Mount

	Bind Mount	Loop Mount
Source	Existing directory	Filesystem image
Command	<code>mount --bind</code>	<code>mount -o loop</code>
New filesystem	No	Filesystem already in image
Main purpose	Alternate access path	Mount filesystem image
Device required	No physical device	Loop device

Why is Bind Mount Useful?

Bind mounts are useful for:

- Providing an alternate path to existing data.
- Container and namespace environments.
- Making selected directories available inside another filesystem tree.
- Testing and system administration.
- Demonstrating Linux mount namespaces.

FOSD Connection

Bind mounts demonstrate how the operating system can modify the **namespace through which files are accessed** without copying the underlying data.

Common Problem: Target is Busy

If:

```
sudo umount ~/mountpoint
```

returns:

```
target is busy
```

first leave the directory:

```
cd ~  
sudo umount ~/mountpoint
```

If required, identify processes:

```
fuser -vm ~/mountpoint
```

Key Takeaways

- ① `mount --bind` provides another path to an existing directory.
- ② It does not copy files.
- ③ Both paths access the same underlying data.
- ④ `findmnt` can be used to inspect the mount.
- ⑤ `umount` removes the bind relationship.
- ⑥ Unmounting does not delete the source files.

Core Concept

Bind mount = another access path, not another copy.

Classroom Activity

Ask students to perform:

- 1 Create `~/source` and `~/mountpoint`.
- 2 Create a file inside `~/source`.
- 3 Bind mount `~/source` onto `~/mountpoint`.
- 4 Read the file through `~/mountpoint`.
- 5 Create a second file through `~/mountpoint`.
- 6 Verify that it appears in `~/source`.
- 7 Unmount the bind mount.
- 8 Verify that the source files still exist.

Question

If no files are copied, why can the same file be accessed through two different paths?

```
pmj@charusat:~/os/mount/source$ ls
source.txt
pmj@charusat:~/os/mount/source$ cat source.txt
Source file
pmj@charusat:~/os/mount/source$ cd ../mountpoint/
pmj@charusat:~/os/mount/mountpoint$ ls
hello.txt
pmj@charusat:~/os/mount/mountpoint$ cat hello.txt
hello
pmj@charusat:~/os/mount/mountpoint$ cd ..
pmj@charusat:~/os/mount$ sudo mount --bind source/ mountpoint/
pmj@charusat:~/os/mount$ cd mountpoint/
pmj@charusat:~/os/mount/mountpoint$ ls
source.txt
pmj@charusat:~/os/mount/mountpoint$ cat source.txt
Source file
pmj@charusat:~/os/mount/mountpoint$ cd ..
pmj@charusat:~/os/mount$ sudo umount mountpoint
pmj@charusat:~/os/mount$ cd mountpoint/
pmj@charusat:~/os/mount/mountpoint$ ls
hello.txt
pmj@charusat:~/os/mount/mountpoint$ cat hello.txt
hello
```

- Silberschatz, Galvin, Gagne, *Operating System Concepts*, 10th Edition.
- William Stallings, *Operating Systems: Internals and Design Principles*, 9th Edition.
- Linux manual pages: `mount`, `umount`, `findmnt`.